

Temperature Monitoring & Visualization System

Technical Task Presentation - How I Approached the Task, and the Results

Interview Walkthrough • Embedded Software Architecture • C bare-metal + C++ OOP implementations • GitHub repository

Presented by Rajarajeswari Mani

ZEISS – Software Architect, Embedded Control Systems

Date: 24/08/2026

AGENDA

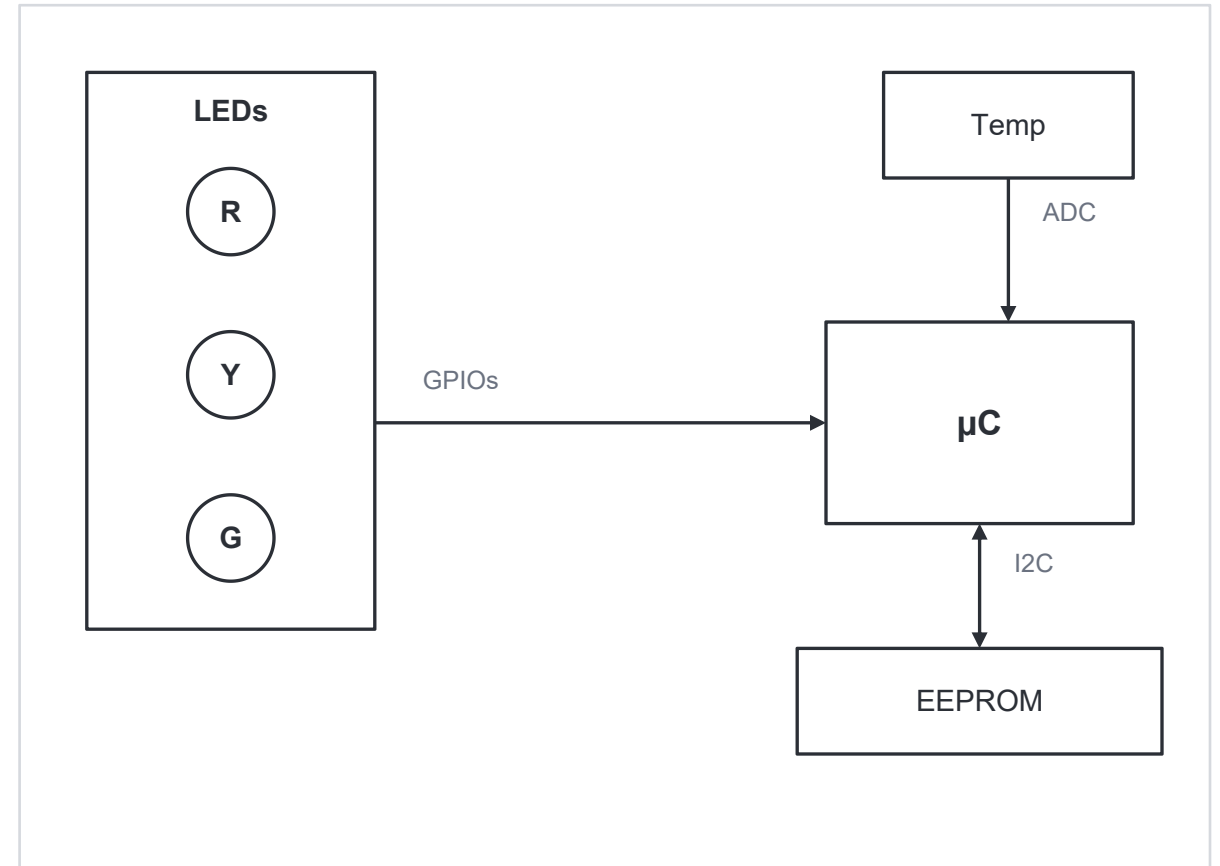
What I'll Walk You Through

- What the task actually asks for, and what I need to deliver
- Requirements — System, Hardware, Temperature/LED, Software & Error Handling
- My approach — the process I followed, step by step
- Architecture decisions and why I made them
- The C implementation — what it does, how it's structured
- The C++ / OOP implementation — how it differs, and why
- Testing — how I verified the requirements, and the results
- Requirements coverage recap
- GitHub repository

INTRO — WHAT IS THE TASK

Task Description

- Architect and design software for a bare-metal embedded system used in a temperature monitoring and visualization device
- The device shows the operating condition on 3 LEDs — OK (green), Warning (yellow), Critical (red)
- A temperature sensor feeds an ADC; the microcontroller reads it, classifies it, and drives the LEDs via GPIO
- An EEPROM (I2C) holds configuration: which hardware revision is fitted, and a serial number placeholder
- This is the block diagram given in the task spec — the starting point for everything I designed:



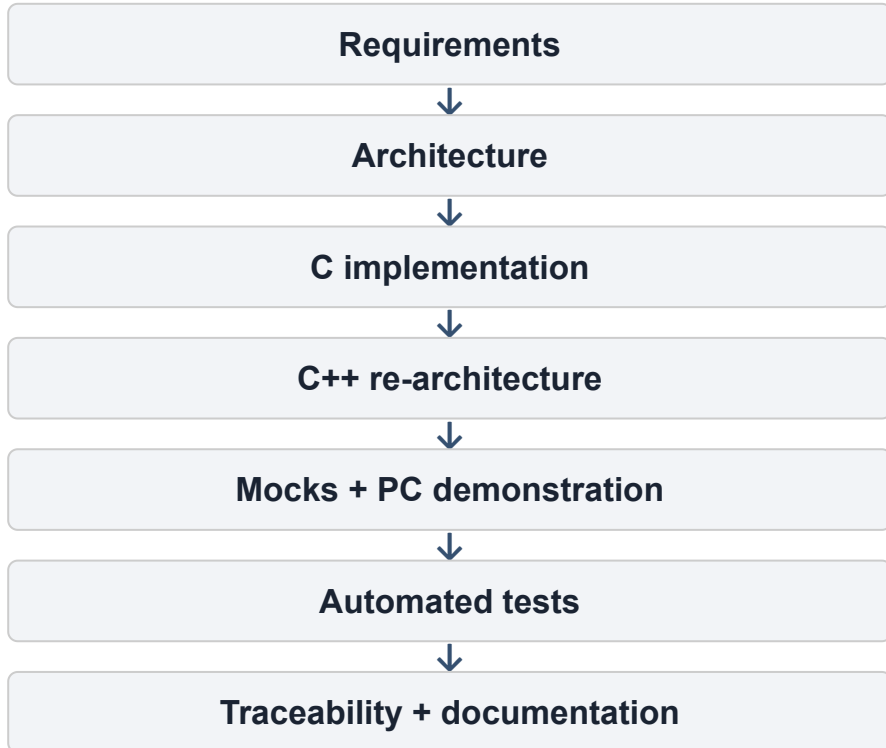
INTRO — DELIVERABLES

What I Need to Do

- Sample temperature via ADC every 100 μ s, very low jitter, and drive 3 GPIO LEDs from fixed thresholds
- Support two hardware revisions with different sensor resolution, selected via EEPROM — only one active at a time
- Write the software in C for a bare-metal target, demonstrable on a PC with mocked hardware — no real hardware or emulator required
- Define ISR function(s) even though they don't need to be functional in the PC demo
- Deliver a second implementation in C++ using OOP
- Document the architecture as a diagram (.pdf / .png / .svg)
- Publish everything (code + diagrams) to a public GitHub repository, link shared $\geq$ 24h before this interview

Engineering Approach

My development flow was:



Why architecture before implementation

I designed the architecture before implementing the application logic so that each major requirement had an identifiable software component and verification activity. That ordering is what makes the requirements traceability chain (Section 17) possible — every requirement maps to a named block, and every block maps to a test.

Requirements — My Starting Point

I converted the task specification into explicit, traceable requirements.

System

- Temperature monitoring and 3-LED visualization
- PC demonstration with mocked hardware
- C and C++ implementations
- Architecture documentation and public repository

Hardware

- ADC, GPIO, timer and I2C EEPROM
- Rev-A = 1°C/digit
- Rev-B = 0.1°C/digit
- Only the fitted sensor type is active

Software

- 100µs sampling
- Minimal ISR
- Conversion, classification and LED control outside the ISR
- EEPROM configuration
- Hardware abstraction and PC mocks

REQUIREMENTS 1/4

System Requirements

ID	Requirement
SYS-1	Monitor temperature and visualize status via 3 LEDs (OK / Warning / Critical).
SYS-2	Architecture documented as a diagram in .pdf / .png / .svg.
SYS-3	Demonstrable on a PC using mocked HW interfaces — no target hardware or emulator required.
SYS-4	Two implementations: C (procedural/bare-metal) and C++ (OOP).
SYS-5	Project + diagrams published to a public GitHub repository, link shared ≥24h before the interview.

REQUIREMENTS 2/4

Hardware Requirements

ID	Requirement
HW-1	Temperature sensor connected to the MCU via an ADC channel.
HW-2	Three LEDs (Green/Yellow/Red) connected via GPIO pins.
HW-3	EEPROM provides hardware revision + serial number; mocked for the PC demo.
HW-4	Two hardware revisions supported: Rev-A = 1 °C/digit, Rev-B = 0.1 °C/digit. Only one active at a time.
HW-5	Timer peripheral generating a periodic interrupt at 100μs to drive sampling.

REQUIREMENTS 3/4

Temperature Conversion & LED Requirements

ID	Requirement
TEMP-1 / TEMP-2	Rev-A: raw = °C directly. Rev-B: raw at 0.1°C resolution (raw 100 = 10°C).
TEMP-3 / TEMP-4	Conversion uses the EEPROM-configured revision; an invalid revision is rejected.
LED-1 / LED-2 / LED-3	Green: $5^{\circ}\text{C} \leq T < 85^{\circ}\text{C}$. Yellow: $85^{\circ}\text{C} \leq T < 105^{\circ}\text{C}$. Red: $T \geq 105^{\circ}\text{C}$ or $T < 5^{\circ}\text{C}$.
LED-4 / LED-5	Exactly one LED lit at a time; all three boundaries handled explicitly.

REQUIREMENTS 4/4

Software & Error Handling Requirements

ID	Requirement
SW-1 / SW-2	Sample every 100μs, very low jitter; ISR does minimal deterministic work only (read, latch, flag).
SW-3 / SW-4	Convert raw→°C per hardware revision; read revision + serial from EEPROM at startup.
SW-5	ISR function(s) defined/implemented, even if not wired to a real interrupt in the PC demo.
SW-6 / SW-7 / SW-8	C bare-metal + C++ OOP, both with mocked ADC/GPIO/EEPROM interfaces for the PC demo.
ERR-1 / ERR-2 / ERR-3	Invalid hardware revision and NULL sensor pointer rejected; no silent fallback to a wrong resolution.

PROCESS

My Approach

1

Read the spec closely, extracted requirements

Split into System / Hardware / Software requirements, each given an ID (SYS-x, HW-x, SW-x) — just covered — so every line in the task PDF traces to something concrete.

2

Designed the architecture before writing code

System static view, hardware static view, software static view (class diagram), software dynamic view (sequence diagram) — drawn in Mermaid

3

Built the C implementation first

Plain procedural C: HAL layer (ADC/GPIO/EEPROM), sensor driver, classifier, LED controller, minimal-work ISR, mocked hardware for PC demo.

4

Re-architected the same behavior in C++ (OOP)

Interfaces + dependency injection for hardware abstraction, Strategy pattern for the Rev-A/Rev-B resolution difference, one orchestrator class.

5

Wrote automated tests for both, tied to requirements

Each test case maps to a specific requirement ID — boundary conditions, revision selection, EEPROM round-trip, ISR presence.

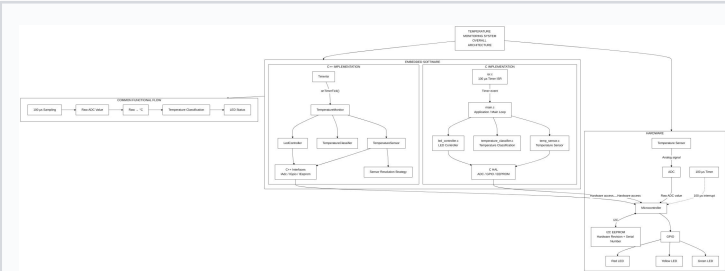
6

Documented and packaged for review

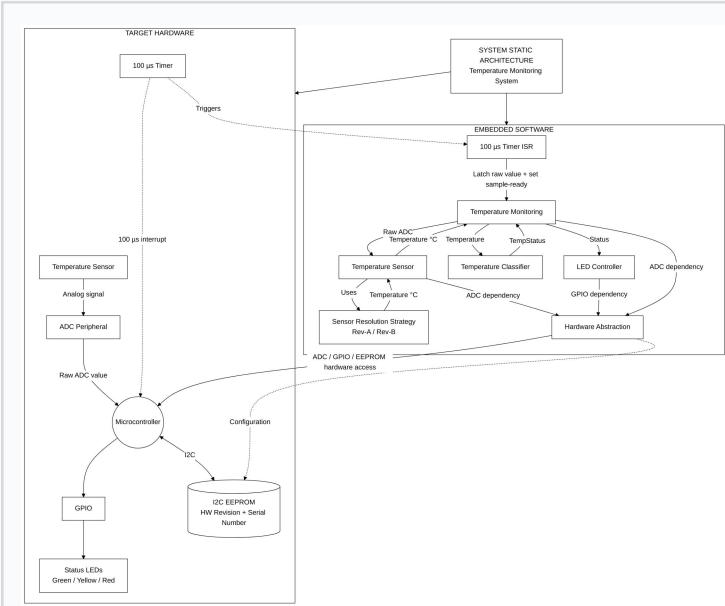
This presentation, README with build instructions, requirements & test-plan docs, all pushed to a public GitHub repo.

DESIGN 1/2

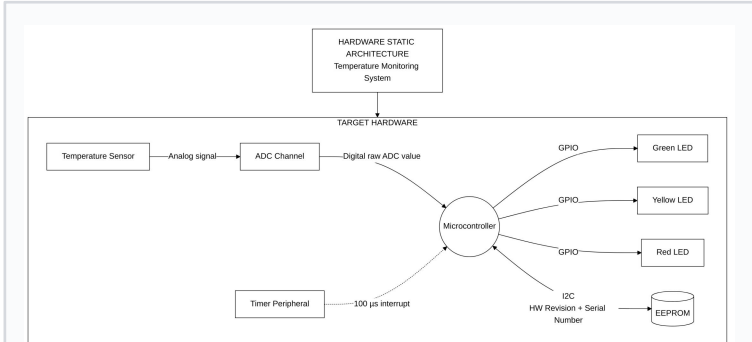
Architecture, At a Glance — System & Hardware



Overall architecture



System static

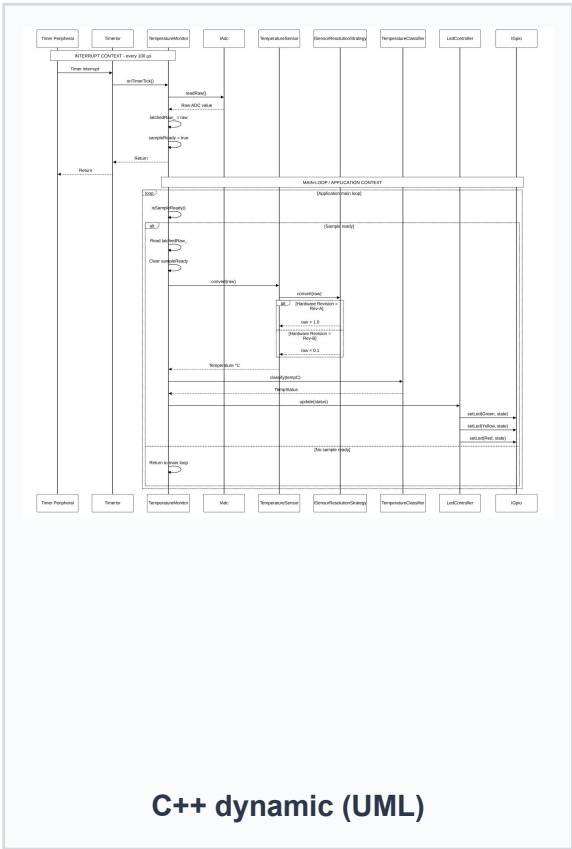
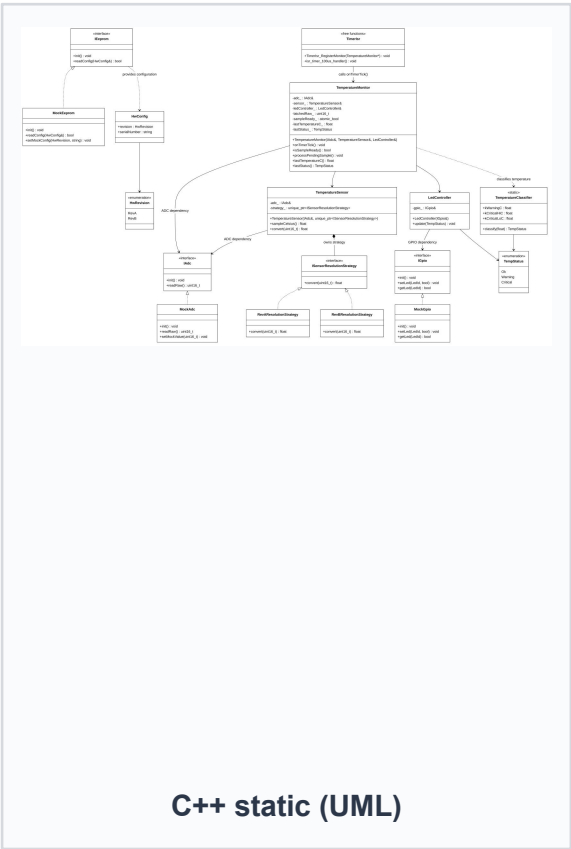
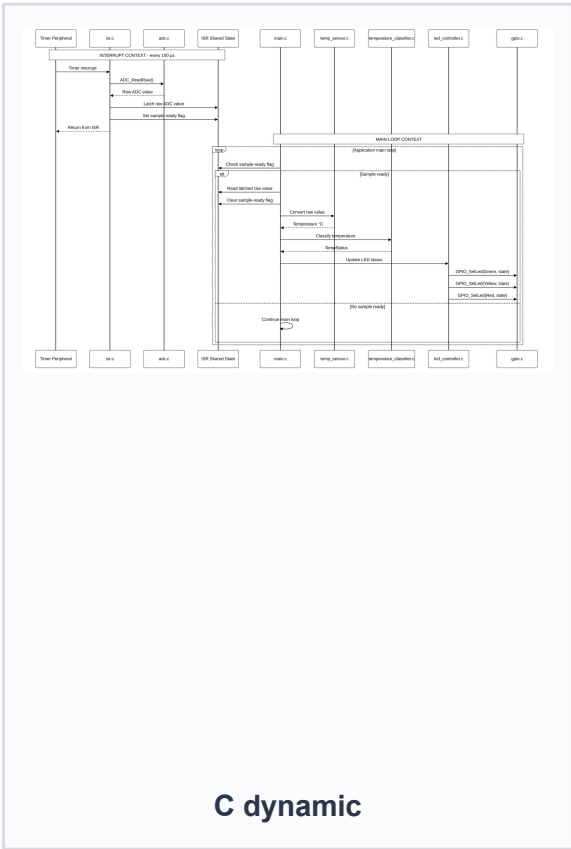
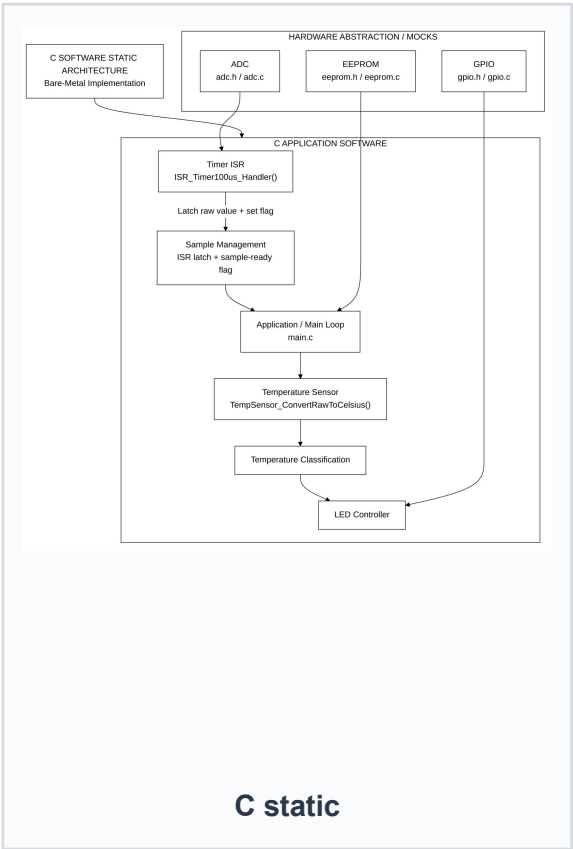


Hardware static

All seven diagrams, plus a block-by-block explanation of each, are in docs/diagrams.md in the repo.

DESIGN 2/2

Architecture, At a Glance — Software (C vs C++)



Same deferred-processing pattern in both languages: a minimal-work ISR latches the raw value; conversion, classification, and LED update happen in the main loop.

DESIGN

Key Design Decisions

- Minimal-work ISR: the 100 μ s interrupt only latches the raw ADC value and sets a flag — no float math, no I/O — so the interrupt has constant, predictable execution time
- Deferred processing: conversion, classification, and LED updates happen in the main loop, outside interrupt context, right after the flag is set
- Strategy pattern for sensor resolution: Rev-A/Rev-B is a swappable object (C++) or a simple selector (C), chosen once from EEPROM at startup — not an if/else sprinkled through the code
- Hardware abstraction + dependency injection: IAdc / IGpio / IEeprom interfaces mean the exact same application logic runs against mocks on a PC or real peripherals on target

DESIGN RATIONALE

Why Keep the ISR Minimal?

- 100µs period = a 10 kHz sampling event — whatever runs in the ISR runs 10,000 times a second
- A short, constant-time ISR reduces interrupt execution time and timing variability
- Avoiding long operations (float math, I/O, EEPROM access) in interrupt context protects the real-time sampling path
- Deferred processing makes the split between “time-critical” and “application-level” work explicit in the architecture, not just a comment in the code
- This also makes the design easier to reason about and verify — the ISR has exactly one job, so it's easy to review and easy to test

RESULTS — C

The C Implementation

- Plain, procedural C11 — structs and free functions, no function pointers or opaque handles
- `adc.h` / `gpio.h` / `eeeprom.h` / `temp_sensor.h` / `led_controller.h` / `isr.h` / `config.h`, each mocked for the PC demo
- `main.c` drives a synthetic temperature ramp through both hardware revisions (with their own EEPROM serial numbers) and prints the resulting LED state
- Builds with `gcc -std=c11 -Wall -Wextra -Wpedantic` — zero warnings

temp_monitor_demo – actual console output

```
$ make && ./temp_monitor_demo
=== Hardware Revision: Rev-A (1 degC/digit) | Serial: ABC1234 ===
[GPIO] LED GREEN -> ON
[OK (low-normal)] raw=20 temp= 20.0C status=OK
[GPIO] LED YELLOW -> ON
[WARNING (>=85C)] raw=90 temp= 90.0C status=WARNING
[GPIO] LED RED -> ON
[CRITICAL (>=105C)] raw=110 temp=110.0C status=CRITICAL
[CRITICAL (<5C, cold)] raw=2 temp= 2.0C status=CRITICAL
[back to OK] raw=25 temp= 25.0C status=OK
=== Hardware Revision: Rev-B (0.1 degC/digit) | Serial: XYZ9876 ===
[OK] raw=200 temp=20.0C ... same sequence, x0.1 scale
```


RESULTS — C++

The C++ / OOP Implementation

- C++17, interfaces + dependency injection: IAdc, IGpio, IEeprom, ISensorResolutionStrategy
- TemperatureSensor, TemperatureClassifier, LedController, TemperatureMonitor (orchestrator)
- RevAResolutionStrategy / RevBResolutionStrategy implement the Strategy pattern for the two hardware revisions
- TimerIsr provides a C-linkage trampoline, standing in for a real interrupt vector table entry
- Builds with `g++ -std=c++17 -Wall -Wextra -Wpedantic` — zero warnings, both via Makefile and CMake

temp_monitor_cpp_demo – actual console output

```
$ make && ./temp_monitor_cpp_demo
=== Hardware Revision: Rev-A (1 degC/digit) | Serial: ABC1234 ===
[GPIO] LED GREEN -> ON
[OK (low-normal)] raw=20 temp= 20.0C status=OK
[GPIO] LED YELLOW -> ON
[WARNING (>=85C)] raw=90 temp= 90.0C status=WARNING
[GPIO] LED RED -> ON
[CRITICAL (>=105C)] raw=110 temp=110.0C status=CRITICAL
[CRITICAL (<5C, cold)] raw=2 temp= 2.0C status=CRITICAL
[back to OK] raw=25 temp= 25.0C status=OK
Identical behavior to the C demo, incl. Rev-B – same
demo points, different implementation paradigm.
```

TESTING

How I Verified It

- Every requirement (SYS/HW/TEMP/LED/SW/ERR-x) has at least one test case tied to it in docs/TEST_PLAN.md
- Both C and C++ have a self-contained test suite — no external framework to install, just `make test`
- Covers: revision conversion scaling, all three threshold boundaries, exactly-one-LED-lit invariant, EEPROM-driven revision selection, serial round-trip, ISR presence, invalid revision / NULL pointer rejection
- Also run automatically on every push via GitHub Actions (.github/workflows/ci.yml)

C — make test (11 checks)

```
$ cd c && make test
[TEST 1] TC-01 Rev-A raw->Celsius (x1) ... PASS
[TEST 4] TC-04 WARNING/CRITICAL boundary at 105C ... PASS
[TEST 10] TC-11 invalid hardware revision rejected ... PASS
[TEST 11] TC-12 NULL sensor pointer rejected ... PASS
Passed: 11   Failed: 0   -> ALL TESTS PASSED
```

C++ — make test (10 checks)

```
$ cd cpp && make test
[TEST 1] TC-01 Rev-A raw->Celsius (x1) ... PASS
[TEST 4] TC-04 WARNING/CRITICAL boundary at 105C ... PASS
[TEST 9] TC-10 ISR entry point (onTimerTick) ... PASS
[TEST 10] TC-11 invalid hardware revision rejected ... PASS
Passed: 10   Failed: 0   -> ALL TESTS PASSED
```

SCOPE & HONESTY

What the PC Demo Proves — and Doesn't

Proven on the PC demo

- Functional behavior of the whole pipeline
- C and C++ produce equivalent results
- Mocked hardware integration (ADC / GPIO / EEPROM)
- Rev-A / Rev-B conversion correctness
- Temperature classification and boundaries
- LED behavior — exactly one lit at a time
- ISR / application separation (the code structure itself)

Still needs target hardware

- Actual timer accuracy
- True 100 μ s period on real silicon
- Real interrupt jitter
- ADC sampling timing
- Real ISR execution time
- Timing margin under real system load

RESULTS

Results Summary

21 / 21

Automated tests passing
(11 C + 10 C++)

0

Compiler warnings
(-Wall -Wextra -Wpedantic)

2

Working implementations
(C procedural, C++ OOP)

7

Architecture diagrams
(verified)

- Both demo programs run end-to-end on a plain PC — no target hardware, no emulator
- Both build clean via their documented commands (make, and CMake for C++) — verified from a fresh clone, not just on my machine
- Every requirement in the task PDF is traced to a design element and at least one test case — see docs/REQUIREMENTS.md and docs/TEST_PLAN.md
- All Architecture Diagrams were re-checked and verified

TRACEABILITY RECAP

Closing the Loop — Every Requirement Accounted For

Category	Status
System (SYS-1..5)	Done — architecture documented, C+C++ delivered, PC-mocked demo; publishing to GitHub is the one remaining step, covered on the next slide
Hardware (HW-1..5)	Done — ADC, GPIO, I2C EEPROM, timer, two revisions all represented in the abstraction layer + diagrams
Temperature & LED (TEMP-1..4, LED-1..5)	Done — Rev-A/Rev-B conversion, all three thresholds (5, 85, 105°C), exactly-one-LED invariant, all boundary-tested
Software (SW-1..8)	Done — 100µs minimal-work ISR, revision-aware conversion, EEPROM config, mocked HW, both language implementations
Error Handling (ERR-1..3)	Done — invalid hardware revision and NULL sensor pointer rejected, no silent fallback

Requirement → Architecture → Implementation → Test

Full ID-by-ID traceability table is in docs/REQUIREMENTS.md in the repo.

GITHUB

Repository

<https://github.com/RajarajeswariMani/Temperature-Monitoring-and-Visualization-System>

Public repository

- Public repository — code (c/, cpp/), diagrams (docs/diagrams/), docs (README, REQUIREMENTS, TEST_PLAN), and this presentation, all in one place
- Builds from a clean clone with the documented commands (make in c/ and cpp/, or CMake for C++)
- GitHub Actions runs the full build + test suite on every push, so the green checkmark is live proof it still works
- Link shared with you $\geq$ 24h ahead of this interview, per the task's own requirement

CLOSING

Key Engineering Takeaways

1

Keep the 100 μ s ISR minimal and deterministic

One HAL read, two variable writes — nothing else runs in interrupt context.

2

Separate real-time acquisition from application processing

The sample-ready flag is the explicit hand-off point between the two.

3

Abstract hardware so the application can run on a PC

IAdc / IGpio / IEeprom (and their C equivalents) mean no target hardware is required to demonstrate or test the logic.

4

Keep revision-specific conversion isolated

Strategy pattern (C++) / single conversion function (C) instead of Rev-A/Rev-B conditionals spread through the code.

5

Implement the functional baseline in C first, then re-architect in C++

Same behavior, two paradigms — makes the OOP benefits concrete rather than theoretical.

6

Verify behavior through explicit requirements and tests

Every requirement ID traces to a design element and at least one automated or reviewed test case.

Thank You

Happy to walk through any part of the code, diagrams, or test results in detail.